PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR
Carrera de Ingeniería en Ciberseguridad - Desarrollo Basado en Plataformas

EVIDENCIAS - TAREA 1 SEMANA 8

Integración, despliegue y defensa de la plataforma web segura de gestión de incidentes

Estudiante	Javier Elías Arteaga Pinargote
Docente	Damián Aníbal Nicolalde Rodríguez
Periodo	2026-02
URL de producción	https://dbp-incidentes.onrender.com
Repositorio	https://github.com/javieroel/DBP (etiqueta v2.0.0)
Versión desplegada	2.0.0 - Render (región Virginia), PostgreSQL 16
Fecha	24/10/2026

Aviso para la revisión: la plataforma usa el plan gratuito de Render. Si nadie la visita durante 15 minutos el servicio se suspende y la primera carga tarda cerca de un minuto, mostrando la pantalla de la Figura E3. No es un error: basta con esperar sin recargar.

Ninguna captura muestra contraseñas, tokens ni valores de variables de entorno. Los datos personales (correo e IP pública del estudiante) aparecen enmascarados o difuminados.

1. Matriz de requisitos y evidencias

Cada requisito de la consigna se relaciona con la evidencia que lo demuestra y con el lugar del repositorio donde está implementado o documentado.

URL funcional	Figuras E2, E3, E4; 39/39 contra la URL (E6)	render.yaml
CRUD persistente	E6 (crear, consultar, actualizar, eliminar; 404 tras borrar); E9	src/services, src/models, migrations/
Autenticación	E6 (login por rol, token falso, alg none: 401); E14	src/middleware/auth.js, src/services/authService.js
Autorización	E6 (403 por rol); E8 (vista lector) frente a E9 (admin)	src/routes/incidentRoutes.js
Validación	E6 (400, XSS, inyección SQL, 409); E10	src/middleware/sanitize.js, src/models, js/controller.js
HTTPS	E4, E5 (HSTS, CSP upgrade-insecure-requests)	Render (TLS), app.js
Configuración por entorno	E1 (variables sin valores), E4 (entorno production)	src/config/config.js, src/config/security.js, .env.example
Logs	E11, E12, E13 (JSON, reqId, eventos de seguridad, correo enmascarado)	src/utils/log.js, src/middleware/logger.js
Health check	E4 (versión, commit, entorno, base)	app.js (/api/salud), render.yaml (healthCheckPath)
Pruebas finales	E6 (39/39 en producción), E7 (unitarias y ESLint)	scripts/pruebas.js, tests/, reportes/
Repositorio versionado	Etiqueta v2.0.0; historial por entrega	GitHub
Documentación técnica	docs/DOCUMENTACION_TECNICA.md	Repositorio
Manual de usuario	docs/MANUAL_USUARIO.md (figuras M0 a M6)	Repositorio
Presentación y defensa	Presentación de 13 diapositivas; guion en docs/GUION_DEFENSA.md	Video de defensa

2. Despliegue en producción

La producción se publicó en Render y no en el servidor propio del estudiante, que aloja otros servicios con información personal: así el tráfico de internet nunca llega a la red doméstica. La infraestructura está descrita como código en render.yaml; la base de datos no tiene acceso público (ipAllowList vacía) y los secretos se generan o ingresan en el panel, nunca en el repositorio.

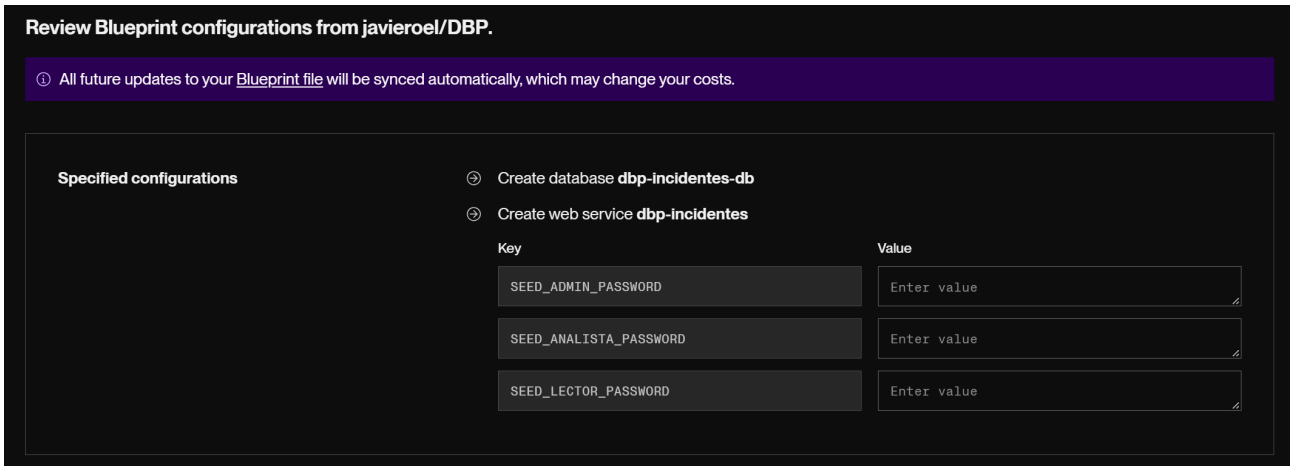


Figura E1. Blueprint de Render: crea la base y el servicio desde render.yaml y solicita las contraseñas iniciales sin valores precargados.

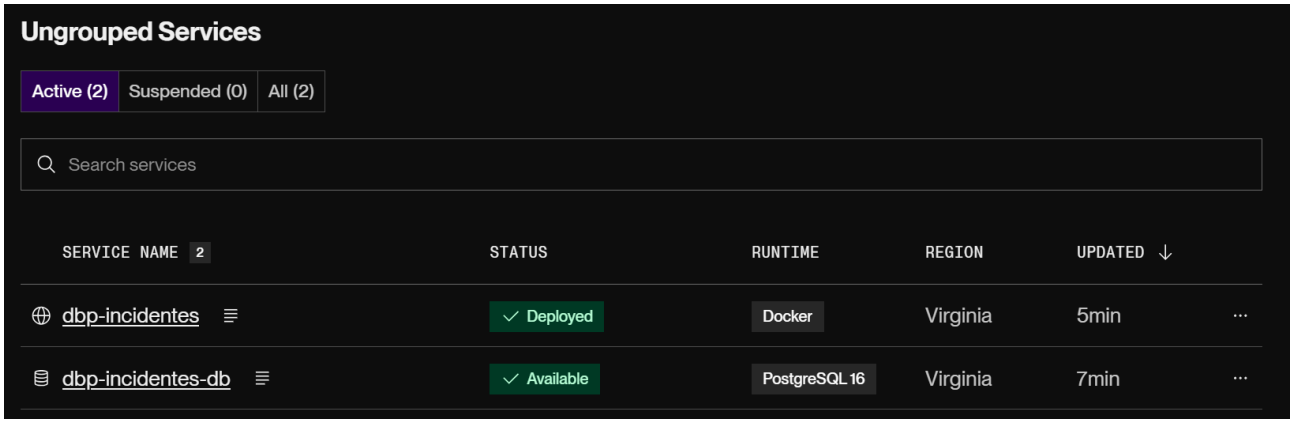


Figura E2. Servicio web (Docker) desplegado y base PostgreSQL 16 disponible, ambos en Virginia.

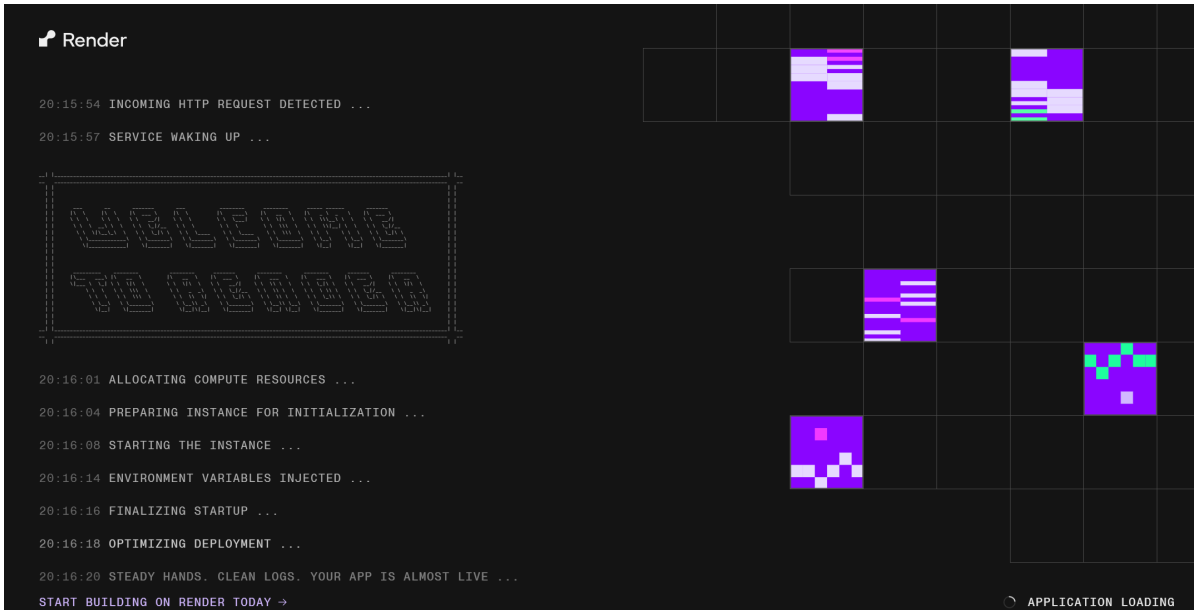


Figura E3. Pantalla que muestra Render mientras el servicio despierta tras 15 minutos sin tráfico (unos 30 segundos en esta captura).

3. HTTPS, cabeceras de seguridad y health check

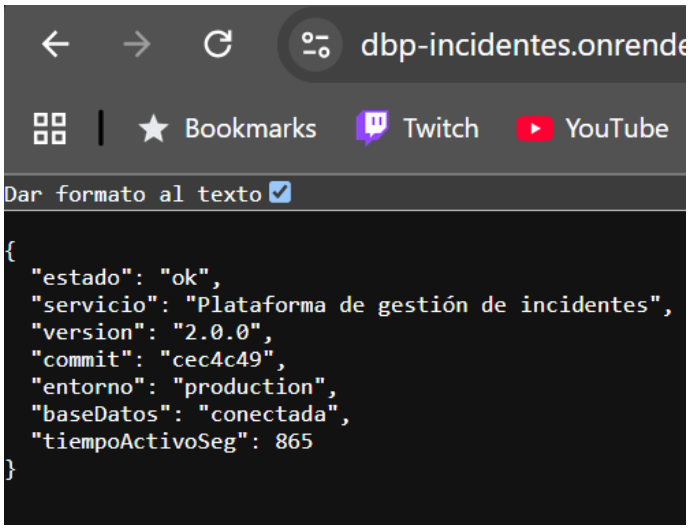


Figura E4. /api/salud en producción: estado ok, versión 2.0.0, commit desplegado, entorno production y base conectada. Responde 503 si la base no está disponible.

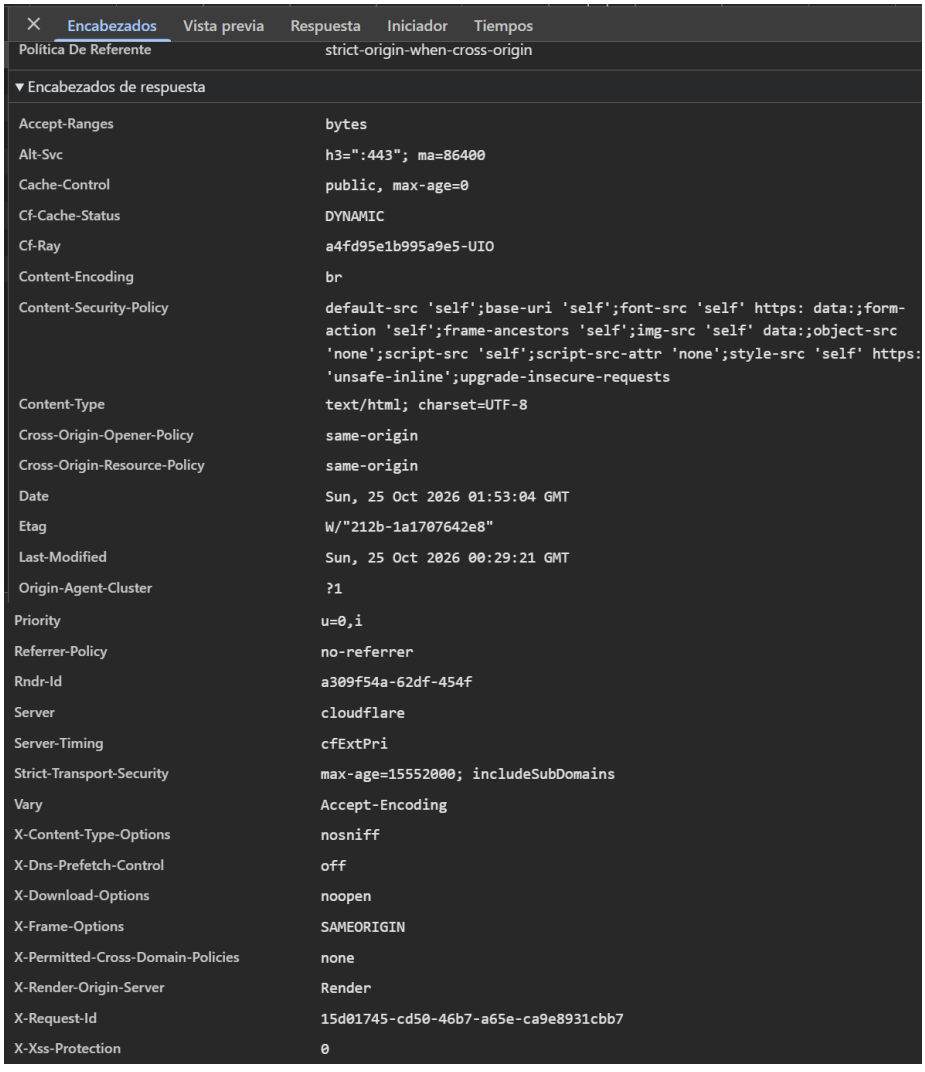


Figura E5. Encabezados de respuesta en producción: Content-Security-Policy, Strict-Transport-Security (HSTS), X-Content-Type-Options, X-Frame-Options, Referrer-Policy y X-Request-Id. No aparece X-Powered-By.

4. Pruebas finales

El script de integración (npm run probar) se ejecutó contra la URL pública. Las contraseñas se ingresaron con read -s y no quedaron en el historial. La salida completa está en reportes/pruebas-produccion.txt.

Autenticación	8	200, 401, 400	8/8
Acceso sin token o con token falso (alg none, otra clave, rol alterado)	6	401	6/6
CRUD con los permisos correctos	5	201, 200	5/5
Permisos insuficientes	4	403	4/4
Datos inválidos, regla de negocio, XSS, inyección SQL, asignación masiva	10	400, 409, 404, 201, 200	10/10
CORS	2	403, 200	2/2
Eliminación y comprobación	4	204, 404	4/4



Figura E6. Pruebas de integración contra https://dbp-incidentes.onrender.com: 39 de 39 casos con el código HTTP esperado.



Figura E7. Pruebas unitarias con Jest y análisis estático con ESLint sin problemas.

5. Interfaz, roles y validación en producción

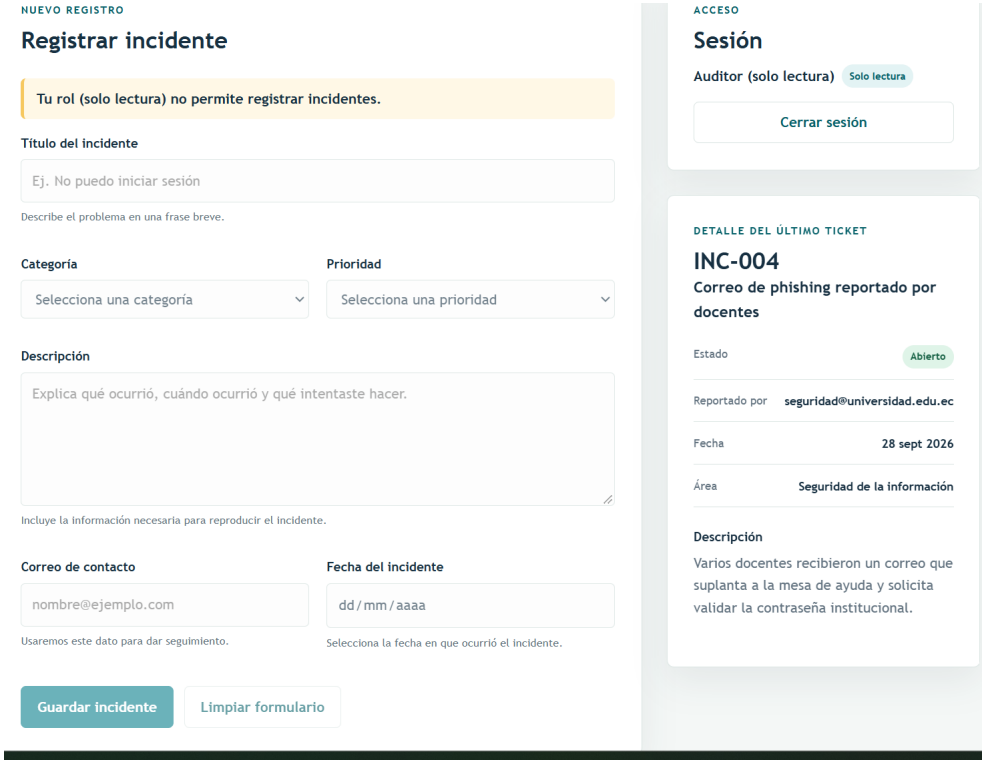


Figura E8. Sesión de lector: aviso de solo lectura, formulario deshabilitado y detalle sin botones de acción. La API además responde 403 si se intenta la acción manualmente.

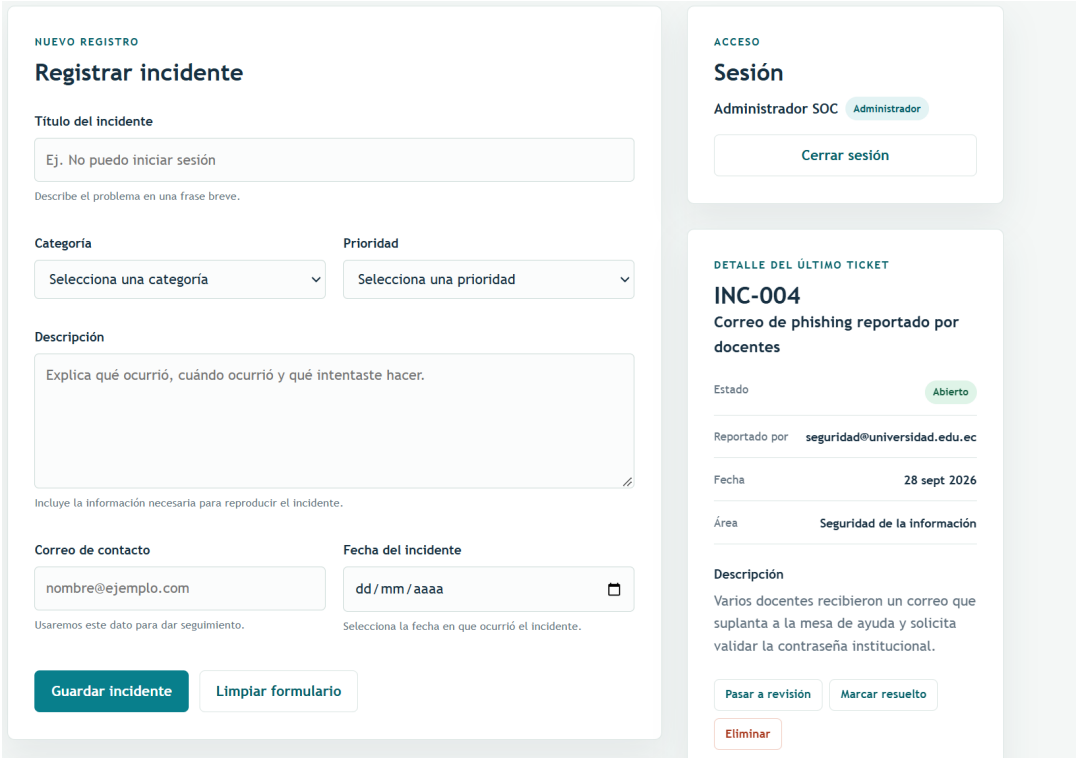


Figura E9. Sesión de administrador: el mismo detalle muestra Pasar a revisión, Marcar resuelto y Eliminar.

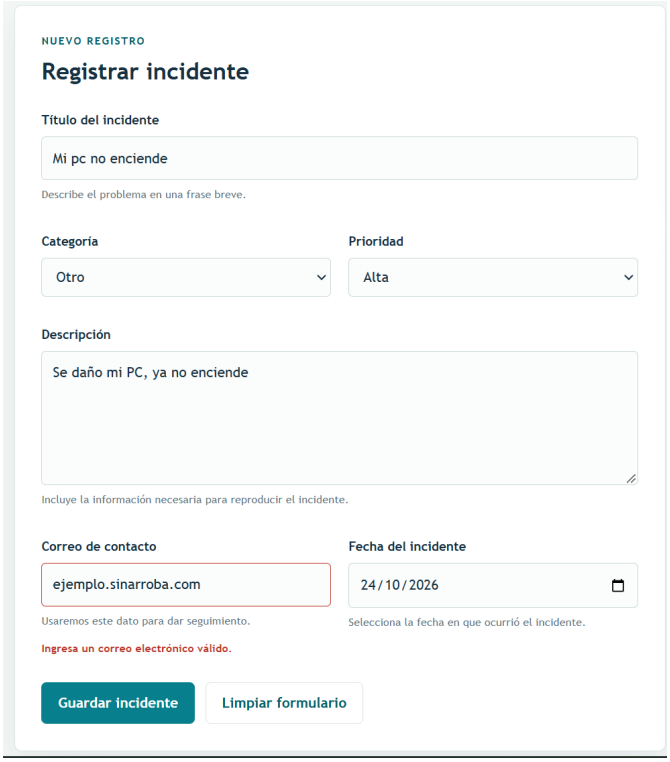


Figura E10. Validación en el navegador: un correo sin @ se marca y no se envía la petición. El servidor valida de nuevo (casos 400 de la Figura E6).

6. Logs, eventos de seguridad y hallazgo en producción

Cada evento se registra como una línea JSON con nivel, fecha e id de solicitud. Los intentos fallidos se marcan como seguridad.login_fallido, el correo se enmascara y nunca se registran contraseñas, tokens ni cabeceras Authorization.

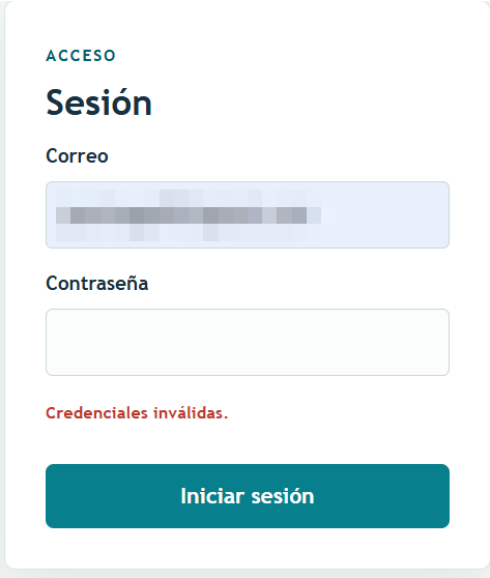


Figura E11. Inicio de sesión con contraseña incorrecta en producción (correo difuminado en la captura).



Figura E12. El mismo intento en los logs de Render: login_fallido con el correo enmascarado y tres eventos con el mismo reqId.

Hallazgo: IP del cliente detrás de tres proxies

En la Figura E12 dos intentos del mismo equipo quedaron con IPs internas distintas: la aplicación confiaba en un solo proxy (valor pensado para Caddy) y en Render hay tres. Con ello el límite de 10 intentos de login se contaba por proxy interno y no por cliente, debilitando la protección contra fuerza bruta. Se agregó al log la cantidad de saltos de X-Forwarded-For (campo xff, sin registrar las IPs), se midió xff = 3 y se fijó TRUST_PROXY = 3, validado para que la aplicación no arranque con true (que permitiría falsificar la IP). El cambio en render.yaml no actualizó la variable del servicio ya creado; se detectó porque el log siguió mostrando una IP interna y se corrigió en el panel.



Figura E13a. Antes: con xff = 3 se registraba una IP interna de Render (10.28.123.5).



Figura E13b. Después de TRUST_PROXY = 3: se registra la IP pública real del cliente (difuminada).

7. Gestión de secretos

Durante la preparación de la entrega se detectó que una copia del archivo .env de desarrollo, con la contraseña real de la base local, había sido compartida como material de contexto en una herramienta externa. Se verificó que el repositorio no estaba afectado (git log --all -- .env sin resultados), se retiró el archivo y se rotó la contraseña del usuario de la base en el servidor de pruebas con \password, comprobando después que /api/salud respondía 200. Producción nació con secretos nuevos: JWT_SECRET generado por Render y contraseñas ingresadas en el panel (Figura E1).

8. Manual de usuario: acceso

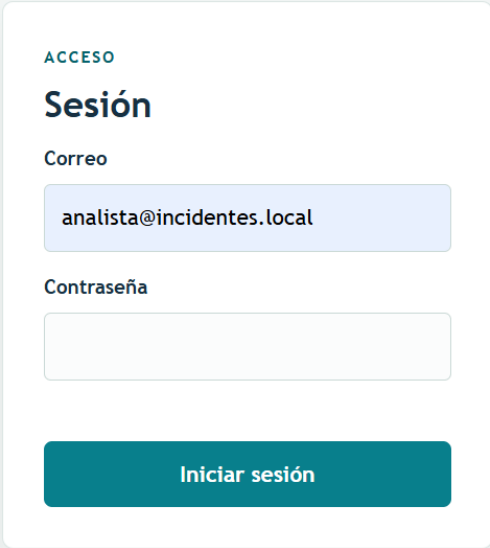


Figura E14. Formulario de inicio de sesión (la contraseña no se muestra). El manual completo está en docs/MANUAL_USUARIO.md.